

GetFed: Accurate, Differentially Private Federated Learning With GAN-Based Data Generation

Hao Bai , Kun He , *Member, IEEE*, Yuqing Li , *Member, IEEE*, Jing Chen , *Senior Member, IEEE*, Haowei Li, Zhongmou Liu, Xuanang Yang , and Ruiying Du 

Abstract—Federated Learning (FL) aims to train neural network models using distributed data resources from multiple clients without sharing raw data. One of the key challenges in FL is non-independent and identically distributed (non-IID) data, which may affect model accuracy. To address this issue, some schemes leverage Generative Adversarial Networks (GANs) to generate virtual data and combine it with the real data to achieve a balanced data distribution. However, there are risks of privacy leakage from the collected virtual data and aggregated gradients. In this paper, we propose GetFed, an accurate and differentially private FL framework with GAN-based Data Generation on non-IID Data. We integrate Differential Privacy (DP) into the GAN training and federated aggregation phases to prevent clients' privacy leakage. To balance privacy and accuracy, we first design a privacy-preserving virtual sample generation algorithm for GAN training that dynamically reduces unnecessary noise as the quality of virtual samples improves. Additionally, we design an adaptive DP-based secure aggregation algorithm that decreases the added noise as the model approaches convergence. Furthermore, we implement a real-virtual ensemble training algorithm, employing an ensemble learning strategy to better mix virtual and real samples for enhanced global model accuracy. This approach ensures clients benefit from both the authenticity of real samples and the balanced data distribution provided by virtual samples, effectively mitigating the data heterogeneity inherent in non-IID scenarios. Extensive experiments demonstrate that compared with state-of-the-art schemes, GetFed improves model accuracy by 6–47% and reduces training time by 50%.

Index Terms—Differential privacy, federated learning (FL), generative adversarial network (GAN), non-IID data.

I. INTRODUCTION

Federated Learning (FL) [1] is currently one of the most promising paradigms in distributed machine learning. Unlike centralized machine learning, FL enables multiple clients

to jointly train a global model by sharing the intermediate results (e.g., gradient updates), while keeping their raw data locally. It has been applied in real-world applications such as cyber security [2], [3], [4], smart healthcare [5], [6], and smart manufacturing [7], [8].

A major challenge in classic FL design is that data across different clients tends to be non-independent and identically distributed (non-IID) [9], which may affect the accuracy of the global model [10], [11]. To handle non-IID data, previous approaches typically involve designing better local loss functions [12], [13], [14], [15], [16] or aggregation methods [17], [18], [19], [20], [21]. However, these schemes are unable to achieve significant improvements in model accuracy due to the fundamental distinction between minimizing local empirical loss and minimizing global empirical loss [22]. Therefore, there is an increasing focus on balancing the distribution by sharing data among clients [9], [23], [24], [25], which is more conducive to improving performance. Since sharing raw data violates the principle of FL, it is common practice to share virtual data generated using methods like Generative Adversarial Networks (GANs) [25], [26], [27]. Virtual data can capture the distribution characteristics of raw data, thus blending them with raw data helps balance the data distribution to address the non-IID problem.

However, there are two security issues in GAN-based FL schemes. First, although intuitively virtual data avoids disclosing information about the real data, researchers have shown that membership inference attacks can still be applied to virtual data to infer whether individual data belongs to the training set [25], [28], [29]. Second, a more common finding is that private information about the raw data can be extracted from the intermediate results uploaded by each client in FL [30], [31], [32], since they are computed based only on the raw data. To solve these problems, a lightweight approach is to incorporate Differential Privacy (DP) [33], [34], [35] into the GAN-based FL scheme (i.e., during both the GAN training and federated aggregation phases). It essentially incorporates clipping and obfuscation through noise injection based on fixed parameters to mislead adversaries by perturbing the results. Nevertheless, the introduced noise significantly reduces model accuracy [16], [36], [37]. There are two main reasons. First, fixed noise injection in traditional DP mechanisms fails to adapt non-IID across clients, leading to either insufficient privacy protection or excessive utility loss. Second, the addition of DP noise exacerbates the mismatch

Received 27 January 2025; revised 5 August 2025; accepted 6 August 2025. Date of publication 11 August 2025; date of current version 4 November 2025. This work was supported in part by the National Key R&D Program of China under Grant 2022YFB3102100, in part by the National Natural Science Foundation of China under Grant 62302343 and Grant 62172303, in part by the Key R&D Program of Hubei Province under Grant 2024BAB018, in part by the Key R&D Program of Shandong Province under Grant 2022CXPT055, and in part by the Wuhan City Joint Innovation Laboratory for Next-Generation Wireless Communication Industry Featuring Satellite-Terrestrial Integration under Grant 4050902040448. (Corresponding authors: Kun He; Yuqing Li.)

The authors are with the Key Laboratory of Aerospace Information Security and Trusted Computing, Ministry of Education, School of Cyber Science and Engineering, Wuhan University, Wuhan 430072, China (e-mail: bh0036@whu.edu.cn; hekun@whu.edu.cn; li.yuqing@whu.edu.cn; chenjing@whu.edu.cn; 2021302181047@whu.edu.cn; lzmstudy2021@whu.edu.cn; whuyxa@whu.edu.cn; duraying@whu.edu.cn).

Digital Object Identifier 10.1109/TDSC.2025.3597635

between local and global optimization objectives in non-IID settings.

In this paper, our goal is to design an accurate and differentially private FL framework. However, this faces three challenges. First, as the model converges, excessive noise is unnecessary. This is because once the training accuracy stabilizes, it is difficult for the attacker to extract any private information from the results [38]. However, the noise of traditional DP is fixed in each round, leading to noise wastage and a subsequent decline in the quality of virtual samples [33]. How to reduce unnecessary noise in the GAN training poses a significant challenge. Second, sharing virtual samples can balance data distribution. Nevertheless, the quality of the virtual samples may still be low compared to the real samples, which could degrade model accuracy if directly mixed with real samples for training [39]. How to better mix virtual and real samples is crucial. Third, clipping intermediate results is a key step in DP. An excessively large value introduces unnecessary noise, while an overly small value leads to frequent gradient clipping, resulting in information loss. Both situations significantly compromise the model accuracy [40]. How to reduce the impact of the clipping process on model accuracy in FL is the third challenge.

To address these challenges, we design an accurate and differentially private FL framework with GAN-based Data Generation on non-IID Data, called Generative Federated Learning (GetFed). First, we propose a privacy-preserving virtual sample generation algorithm that adjusts the noise intensity based on the quality of virtual samples in the GAN training. Because the magnitude of sensitivity in DP is one of the key factors affecting the amount of noise added. As the real and virtual samples become more similar, the model's sensitivity to input data decreases. Consequently, noise can be adaptively reduced based on their similarity. Then, we propose a real-virtual ensemble training algorithm, where two models are trained independently on real and virtual datasets, and aggregated by considering model similarity. This method leverages the advantages of both real and virtual datasets by mixing multiple independently trained models, thereby improving the accuracy of the global model. Finally, we design an adaptive DP-based secure aggregation algorithm to protect gradient updates and minimize the negative impact on model accuracy. In particular, we adjust the clipping factor based on the model convergence progress. As the model converges, the clipping factor decreases moderately, making the noise added to intermediate results also smaller.

Our evaluation considers the testing accuracy and training time on four datasets including MNIST, Fashion-MNIST, CIFAR-10, and SVHN in different non-IID settings. We also investigate the impact of virtual sample privacy parameters and sharing ratio factors, and the effectiveness of adaptive DP-based secure aggregation as well as real-virtual ensemble training. We compare our scheme with FedAvg [1], FedProx [12], DP-SCAFFOLD [13], FedNova [17], FedBN [18], and PRIVATEFL [16].

Our main contributions are summarized as follows.

- We propose GetFed, an accurate and differentially private FL framework via GAN. GetFed effectively addresses the non-IID issue by generating and sharing virtual data among

clients to achieve more balanced data distributions, while also balancing the relationship between virtual and real samples based on the real-virtual ensemble training algorithm.

- We design a privacy-preserving virtual sample generation algorithm to protect clients from membership inference attacks, and employ a novel mechanism to reduce unnecessary noise in the GAN training. Moreover, we choose a subset with a random distribution to hide the real data distribution of the clients.
- We present an adaptive DP-based secure aggregation algorithm that protects gradient updates while mitigating the negative effects of noise on model accuracy in FL, thus effectively balancing privacy and accuracy.
- We conduct extensive experiments to validate the effectiveness of our proposed framework. The results show that GetFed enhances model accuracy by 6–47% and reduces training time by 50% compared to existing schemes.

The remainder of this paper is organized as follows. In Section II, we introduce background on FL, GAN, and DP. In Section III, we detail the proposed GetFed, with its theoretical analysis in Section IV. Experimental results are described in Section V. Section VI describes the related work. We finally conclude this work in Section VII.

II. PRELIMINARIES

We first briefly introduce FL. Then, we review two techniques in our framework, including GAN and DP.

A. Federated Learning

FL aims to establish a global model among multiple clients while preserving the data privacy of each client [1]. Each client trains a local model based on its own dataset. Subsequently, with the assistance of a central server, the intermediate results are aggregated to update a global model, which is distributed to all clients for updating their local models.

Suppose there are N clients and each client i has a local real dataset $D_i^R \triangleq (z_{i,1}, \dots, z_{i,j}, \dots, z_{i,n_i})$, where $z_{i,j} = (x_{i,j}, y_{i,j}) \in \mathcal{X} \times \mathcal{Y}$. $\mathcal{X}$ is the sample space, $\mathcal{Y}$ is the label space, n_i is the size of client i 's dataset, and $n = \sum_{i=1}^N n_i$ is the size of the global dataset, which comprises a total of U classes. We define $l_i(\cdot)$ to be the loss function of client $i \in [N]$, which measure the local empirical risk on its dataset D_i^R , i.e., $l_i(w) = E_{z_{i,j} \in D_i^R} [l_j(w; z_{i,j})]$. The optimization goal of the FL is to find an optimal global model, i.e., $\hat{w} = \arg \min \{l_w = \sum_{i=1}^N \gamma_i l_i(w)\}$, where w is a trained global model and $\gamma_i = \frac{n_i}{n}$ is the weight of the loss function of client i . FL typically employs the Stochastic Gradient Descent (SGD) algorithm to minimize the loss function until a specified number of rounds or model accuracy is achieved. A common problem with FL is the susceptibility to non-IID data, which may affect model accuracy.

B. Generative Adversarial Network

Direct sharing of raw data can solve the non-IID problem, but is forbidden in FL. Therefore, each client utilizes GAN to

generate virtual data and share it to change the data distribution. GAN is a machine learning architecture that consists of two models: a generator ($\mathcal{G}$) and a discriminator ($\mathcal{D}$) [26]. The generator takes low-dimensional random vectors as input and outputs virtual samples, which wants to make the discriminator mistake virtual samples for real samples as much as possible. The discriminator outputs the probability that each input sample is real, which distinguishes between real and virtual samples as accurately as possible. The above process can be described as the following optimization problem:

$$\min_{\mathcal{G}} \max_{\mathcal{D}} \{E_{x \sim p_{\text{data}}} [\log \mathcal{D}(x)] + E_{z \sim p_z} [\log(1 - \mathcal{D}(\mathcal{G}(z)))]\},$$

where x represents samples from the real data distribution p_{data} and z is random vectors from the distribution p_z . $\mathcal{D}(x)$ denotes the probability of considering the input x to be a real sample. $\mathcal{D}(\mathcal{G}(z))$ is the probability of considering the input $\mathcal{G}(z)$ (i.e., a virtual sample generated by the $\mathcal{G}$) to be a real sample.

To allow the generator to generate virtual samples with specified labels, conditional GAN [41], [42], [43], [44], [45], [46] have been proposed. We implement our framework using SNGAN [46], one of the conditional GAN, due to its efficiency and the ability to generate complex samples. There are two main reasons. First, SNGAN's spectral normalization provides inherent gradient control and training stability, which is critical when differential privacy noise injection destabilizes other GAN variants. Second, SNGAN's consistent gradient behavior and reliable generation quality directly enhance the effectiveness of our scheme. However, without implementing a privacy-preserving mechanism, directly sharing virtual samples or gradient updates is easy to leak privacy.

C. Differential Privacy

DP is a provable mathematical framework, which adds carefully designed noise to the data to achieve privacy protection [26]. It ensures that any individual data modification within a dataset will not significantly impact the outputs. Therefore, attackers cannot infer whether individual data exists with the dataset by analyzing the outputs. DP can be defined as follows:

Definition 1 ((ϵ, δ) -DP [47]): A randomized algorithm $\mathcal{A}$ satisfies (ϵ, δ) -differential privacy, if for any two neighboring datasets X and X' that differ in only single data (i.e., X and X' are neighboring if one can be obtained from the other by adding, removing, or modifying a single data), and for any subset of possible outputs O :

$$\Pr[\mathcal{A}(X) \in O] \leq e^\epsilon \Pr[\mathcal{A}(X') \in O] + \delta.$$

The parameter ϵ controls the privacy guarantee, and the parameter δ represents the probability of failure in (ϵ, δ) -DP. The smaller values of ϵ and δ provide stronger privacy protection by applying more noise to the data. The privacy parameter can be interpreted as a privacy budget that determines the amount of privacy loss.

Definition 2 (Sensitivity [47]): For any query function $\mathcal{Q}: X \rightarrow R$, X is the input and R is the output, then the sensitivity

of the function $\mathcal{Q}$ is

$$\Delta \mathcal{Q} = \max_{X, X'} \|\mathcal{Q}(X) - \mathcal{Q}(X')\|_p.$$

Here, $\|\cdot\|_p$ denotes the L_p parametrization. X' means the neighboring dataset that differs from X by only single data. The sensitivity reflects the maximum range of variation of the output of $\mathcal{Q}$ on a pair of neighboring datasets. The smaller the sensitivity, the less noise needs to be added.

Definition 3 (Gaussian mechanism [47]): If a function $\mathcal{Q}$ is computed using the L_2 norm, (ϵ, δ) -differential privacy can be achieved by adding Gaussian noise to the output of the function $\mathcal{Q}$, as follows:

$$\mathcal{A}(X) = \mathcal{Q}(X) + \mathcal{N}(0, \sigma^2 C^2 \mathbf{I}).$$

Here, $\mathcal{N}(0, \sigma^2 C^2 \mathbf{I})$ follows a Gaussian distribution with a mean of 0 and a covariance of $\sigma^2 C^2 \mathbf{I}$, where $\sigma = \sqrt{2 \ln(\frac{1.25}{\delta})} / \epsilon$ is a noise multiplier, C is the clipping factor, and $\mathbf{I}$ represents the identity matrix.

DP has two key properties. One is composability. If multiple algorithms satisfy (ϵ, δ) -DP, the overall privacy guarantee can be determined by the sum of their privacy budgets [33]. For example, if two algorithms have (ϵ_1, δ_1) and (ϵ_2, δ_2) as their respective privacy parameters, running them sequentially would provide $[(\epsilon, \delta) = (\epsilon_1 + \epsilon_2, \delta_1 + \delta_2)]$ -DP. The other is post-processability. If an algorithm satisfies DP, then any operation performed on the output does not result in an additional loss of privacy [33].

Nevertheless, the introduced noise significantly reduces model accuracy, which makes balancing privacy and accuracy still a challenge.

III. DESIGN OF GETFED

We propose GetFed, a privacy-preserving and accurate FL framework on non-IID data. We begin with an overview and the design details of GetFed, including privacy-preserving virtual sample generation, real-virtual ensemble training, and adaptive DP-based secure aggregation.

A. Overview

System Model: Our scheme aims to train a global model for a classification task in FL comprising N clients and a server. Clients possess local datasets, and actively participate in generating virtual samples and training a local model. The server coordinates the entire process. Its tasks include aggregating virtual samples, conducting model aggregation, and updating parameters.

Problem Formulation: Each client i has a local dataset D_i^R . Function $l_i(\cdot)$ represents the loss function of client $i \in [N]$. The optimization goal of the FL is to find an optimal global model, i.e., $\hat{w} = \arg \min \{l_w = \sum_{i=1}^N \frac{n_i}{n} l_i(w)\}$, where w is a trained global model, n_i is the size of client i 's dataset, and $n = \sum_{i=1}^N n_i$ is the size of the global dataset.

Threat Model: Similar to existing FL schemes [16], [48], [49], [50], [51], [52], we assume that an adversary is semi-honest,

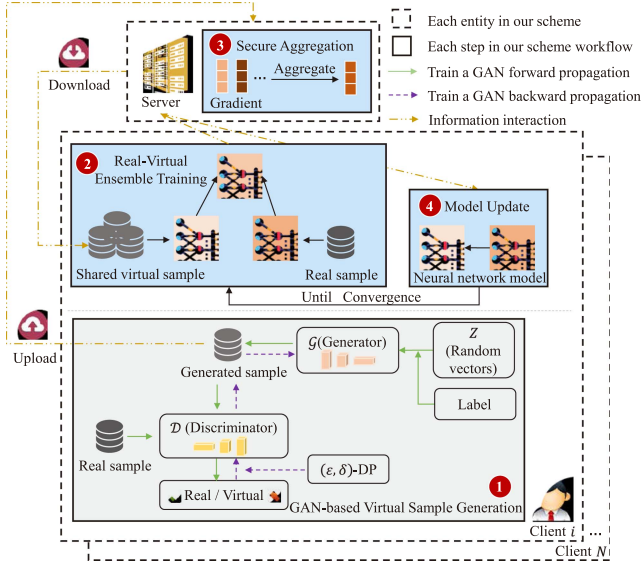


Fig. 1. An overview of GetFed.

meaning it strictly adheres to the algorithms but may be curious about private information of a client's local dataset (e.g., membership).

An adversary may attack other clients of the system by controlling the server (or some clients) during the virtual sample generation phase and the model training phase. In the virtual sample generation phase, an adversary attempts a membership inference attack on a victim client to infer whether individual data belongs to the training datasets [31]. We focus solely on the black-box setting, where the adversary has no access to the victim's discriminator or generator. Specifically, the adversary is unable to obtain parameters to construct a model similar to the victim's, nor does it have access to the loss function or the overall duration of the training process. This is reasonable, as the adversary can only obtain virtual samples from victims without knowing how these samples were generated. Moreover, the adversary is also curious about the label distribution of victims during the virtual sample generation phase because it may infer sensitive information, such as behavioral patterns and habits [53], [54]. In the model training phase, the adversary tries to analyze additional information about private datasets based on gradients [30].

Design Goals: Our goal is to design an accurate and differentially private FL framework on non-IID data. Specifically, we have the following three objectives.

- **Privacy:** During both the GAN training and FL phases, the attacker cannot obtain the client's local private data.
- **Accuracy:** GetFed should aim to minimize model accuracy loss brought by DP.
- **Efficiency:** The virtual sample generation phase can be processed at any time before the FL.

Architecture: Each client trains a GAN satisfying DP based on real data, and then outputs a certain number of virtual samples and uploading them to the server to form a shared dataset. The server distributes the shared dataset to each client

Algorithm 1: GetFed.

Input: Real dataset D_i^R of client i , sharing ratio factor κ , batch size B , initialized model parameters w^0 , pre-processing training rounds T_{pre} , online training rounds T_{online} , privacy budget (ϵ, δ) , noise multiplier σ , clipping factor C .

Output: Global model $w^{T_{online}}$.

// For the server.

- 1: Aggregate D_i^V and distribute the shared virtual dataset $D^V = \sum_{i=1}^N D_i^V$ to all clients.
- 2: **for** $t = 1, \dots, T_{online}$ **do**
- 3: Compute the global gradient $\tilde{g}_t = \sum_{i=1}^N \frac{1}{N} \tilde{g}_{i,t}$.
- 4: $w^{t+1} \leftarrow$ Update the global model based on $\tilde{g}_t$.
- 5: **end for**
- // For client $i \in [N]$.
- 6: Compute $v_i = \kappa |D_i^R|$.
- 7: Select v_i virtual labels from L_i , e.g., $\tilde{y}_1, \dots, \tilde{y}_{v_i}$.
- 8: Generate virtual samples $\tilde{x}_1, \dots, \tilde{x}_{v_i}$ by Algorithm 2.
- 9: Form a virtual dataset $D_i^V = (\tilde{x}_n, \tilde{y}_n)_{n \in [v_i]}$, which is uploaded to the server.
- 10: **for** $t = 1, \dots, T_{online}$ **do**
- 11: Real-virtual ensemble training by Algorithm 3.
- 12: $\tilde{g}_{i,t} \leftarrow$ Add noise to $g_{i,t}$ based on (1).
- 13: Upload $\tilde{g}_{i,t}$ to server.
- 14: Download the model parameters w^t from the server.
- 15: **end for**

to balance the difference of data distribution. First, we design a privacy-preserving virtual sample generation algorithm to reduce unnecessary noise in the GAN training. Note that this can be done in the pre-processing phase. Additionally, we propose a real-virtual ensemble training algorithm, which combines models trained independently on real and virtual datasets. Finally, our adaptive DP-based secure aggregation algorithm safeguards gradient updates, dynamically adjusting based on model convergence progress.

As shown in Fig. 1, GetFed is divided into two phases: the pre-processing virtual sample generation phase (1) and the online model training phase (2, 3, and 4). The online phase is repeated until the predefined number of rounds is reached or until the global model converges. The complete algorithm of GetFed is outlined in Algorithm 1. The workflow is described as follows.

- **GAN-based Virtual Sample Generation (1):** Each client trains a GAN using its local data based on the privacy-preserving virtual sample generation algorithm (lines 6–8), and uploads a specified number of virtual samples to the server (line 9), which aggregates them and distributes (line 1).
- **Real-Virtual Ensemble Training (2):** Each client holds the shared virtual samples and real samples, and performs the real-virtual ensemble training algorithm (lines 11–12).
- **Secure Aggregation (3):** Each client adds noise based on the adaptive DP-based secure aggregation algorithm (lines

12–13). The server aggregates the gradients and updates the global model (lines 3–4).

- **Model Update** (4): Each client downloads the new global model to update its local model, and begins a new training round from Step 2 (line 14).

B. Privacy-Preserving Virtual Sample Generation

We leverage SNGAN in GetFedto generate virtual samples, which is a common virtual sample generation algorithm for specific labels, and further improves the quality of virtual samples. However, it does not deploy privacy protections. Previous studies [25], [28] have indicated that attackers can still conduct membership inference attacks by virtual samples, thereby compromising the clients' data privacy. For example, individuals do not want their images to appear in the virtual images, even if they have been used for training the GAN model. Therefore, we propose a privacy-preserving virtual sample generation algorithm based on DP to safeguard the membership privacy of real training samples.

Based on the SNGAN architecture, each gradient of the discriminator g_b^D is clipped during training: $g_b^D = \frac{g_b^D}{\max(1, \|g_b^D\|/C)}$, where C is the clipping factor. Subsequently, gradients from the batch B are averaged and noise is added $\frac{1}{B}(\sum_{b=1}^B g_b^D + \mathcal{N}(0, \sigma^2 C^2 \mathbf{I}))$. The noise is fixed for each round. However, as the model converges, much of the extra noise is not needed and thus may be wasted, leading to a decrease in the quality of virtual samples [55]. Therefore, we propose a novel method, adjusting the noise magnitude during the GAN training based on the quality of virtual samples.

We utilize the Fréchet Inception Distance (FID) to assess the quality of virtual samples [56]. Specifically, FID employs the intermediate feature representations in the Inception-v3 model for both real samples r_s and virtual samples $\tilde{r}_s$. First, the real and virtual samples are fed forward through the Inception-v3 model to extract features from the second-to-last pooling layer. Then, the FID between these two feature distributions is computed by $\mathcal{F}(r_s, \tilde{r}_s) = \|\mu_1 - \mu_2\|_2^2 + \text{Tr}(\sum_1 + \sum_2 - 2\sqrt{\sum_1 \sum_2})$, where μ_1 and μ_2 represent the mean vectors of features for real and virtual samples, $\sum_1$ and $\sum_2$ represent their covariance matrices. $\text{Tr}(\cdot)$ represents the trace of a matrix, i.e., the sum of the elements on the diagonal of the matrix. A smaller FID indicates a closer resemblance between the feature distributions of real and virtual samples, signifying the higher quality of virtual samples.

Since the magnitude of sensitivity in DP is one of the key factors affecting the amount of noise added. As the real and virtual samples become more similar (i.e., the FID is smaller), the model becomes less sensitive to the input data (i.e., less noise is added). The amount of noise added is directly proportional to the FID: $F(r_s, \tilde{r}_s) \propto \mathcal{F}(r_s, \tilde{r}_s)$. Therefore, we design a formula to calculate $F(r_s, \tilde{r}_s) = \text{normalize}(\mathcal{F}(r_s, \tilde{r}_s))$, where normalize refers to a normalization operator used to transform the value into a probability value, typically ranging between 0 and 1.

Given real samples $(x_1, y_1), \dots, (x_B, y_B)$ and virtual samples $(\tilde{x}_1, \tilde{y}_1), \dots, (\tilde{x}_B, \tilde{y}_B)$, the privacy probability is $F(x_i, \tilde{x}_i)$

Algorithm 2: Privacy-Preserving Virtual Sample Generation.

Input: Real dataset D_i^R of client i , number of virtual samples v_i , virtual labels $\tilde{y}_1, \dots, \tilde{y}_{v_i}$, pre-processing training rounds T_{pre} , privacy budget (ϵ, δ) , noise multiplier σ , clipping factor C , batch size B .

Output: Virtual sample features $\tilde{x}_1, \dots, \tilde{x}_{v_i}$

```

1: Initialize the parameters of the discriminator  $\mathcal{D}$  and the
   generator  $\mathcal{G}$ :  $\Phi_{\mathcal{D}}, \Phi_{\mathcal{G}}$ .
2: for  $t = 1, \dots, T_{pre}$  do
3:   Sample noise vectors from a normal distribution
      $z_1, \dots, z_B$ .
4:   Randomly sample from the real dataset  $D_i^R$  to obtain
      $(x_1, y_1), \dots, (x_B, y_B)$ .
5:   for  $b = 1, \dots, B$  do
6:      $\tilde{x}_b = \mathcal{G}(z_b, y_b)$ .
7:      $g_b^D = \nabla \{\log \mathcal{D}(x_b, y_b) + \log(1 - \mathcal{D}(\tilde{x}_b, y_b))\}$ .
8:      $g_b^D = \frac{g_b^D}{\max(1, \|g_b^D\|/C)}$ .
9:   end for
10:  Calculate adaptive noise  $\zeta \cdot \mathcal{N}(0, \sigma^2 C^2 \mathbf{I})$  per-batch.
11:   $g^D = \frac{1}{B}(\sum_{b=1}^B g_b^D + \zeta \cdot \mathcal{N}(0, \sigma^2 C^2 \mathbf{I}))$ . // add noise.
12:   $\Phi_{\mathcal{D}} = \Phi_{\mathcal{D}} + \eta g^D$ .
13:  for  $b = 1, \dots, B$  do
14:     $g_b^G = \nabla \{\log(1 - \mathcal{D}(\tilde{x}_b, y_b))\}$ .
15:  end for
16:   $g^G = \frac{1}{B} \sum_{b=1}^B g_b^G$ .
17:   $\Phi_{\mathcal{G}} = \Phi_{\mathcal{G}} - \eta g^G$ .
18: end for
19: for  $n = 1, \dots, v_i$  do
20:   Sample noise vectors from a normal distribution
      $z_1, \dots, z_{v_i}$ .
21:    $\tilde{x}_n = \mathcal{G}(z_n, \tilde{y}_n)$ .
22: end for
```

over batch B during the GAN training. To improve performance, we introduce the privacy weight to adjust the magnitude of noise $\zeta = \sum_i^B F(x_i, \tilde{x}_i)/B$. The privacy weight ζ represents the average privacy probability over batch B . ζ changes with the quality of virtual samples and is multiplied by the noise, which avoids the fixed noise in each round and reduces the unnecessary noise. Finally, we incorporate it into the Gaussian noise $\zeta \cdot \mathcal{N}(0, \sigma^2 C^2 \mathbf{I})$. The noise reduction mechanism is based on ζ , which reduces the overall noise without changing the original gradient clipping. Additionally, the real data is only used to calculate ζ , and does not require noise perturbation, so it does not consume the privacy budget.

Virtual Label Selection: Each client needs to select labels to generate virtual samples using the trained GAN. In order to build the shared virtual dataset, not only the features but also the labels should be uploaded to the server. Recall that the label distribution allows inferring sensitive information, therefore, the label selection process must not reveal the real data label distribution [54]. A straightforward approach is to generate the same number of virtual samples for each client. It is independent

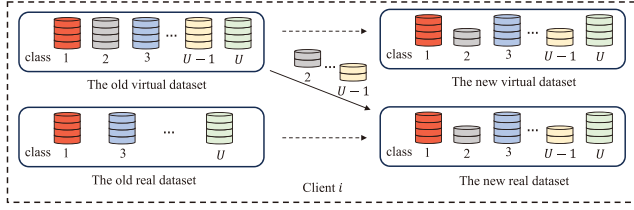


Fig. 2. The example of a method for using real and virtual datasets.

of the local dataset, and is only suitable on the IID data. However, clients may possess samples from only certain classes in the non-IID scenario. The generator may struggle to generate virtual samples.

In order to solve the above issue and ensure the diversity and comprehensiveness of the shared virtual dataset, we propose a random label selection method based on local data volume. The global dataset comprises a total of U classes. For client i , the number of instances in each class on its real dataset is denoted as $n_1^i, \dots, n_U^i$. Let $\hat{n}_1^i, \dots, \hat{n}_U^i$ be random variables, equals $n_1^i, \dots, n_U^i$ plus a normal distribution with mean 0 and variance σ_l^2 , i.e., $\hat{n}_1^i = n_1^i + \mathcal{N}(0, \sigma_l^2), \dots, \hat{n}_U^i = n_U^i + \mathcal{N}(0, \sigma_l^2)$. By perturbing the number of instances for each class, the true number of instances for each class is avoided. In our experiments, we set the value of σ_l^2 to 0.5. The number of virtual labels to be selected for each class is represented as $\tilde{n}_1^i, \dots, \tilde{n}_U^i$, where $\tilde{n}_u^i = \kappa \hat{n}_u^i, u \in [U]$ and κ is sharing ratio factor. The client i is easy to determine all the virtual labels, where $v_i = \sum_{u=1}^U \tilde{n}_u^i$ represents the total number of selected virtual labels. L_i is the set of labels corresponding to all samples in D_i^R . The client i selects v_i virtual labels from L_i , denoted as $\tilde{y}_1, \dots, \tilde{y}_{v_i}$. Next, it generates virtual samples, denoted as $\tilde{x}_1, \dots, \tilde{x}_{v_i}$, corresponding to $\tilde{y}_1, \dots, \tilde{y}_{v_i}$. These virtual samples and labels form a virtual dataset D_i^V , which is uploaded to the server. Finally, the server aggregates them and distributes the shared virtual dataset D^V to all clients.

The privacy-preserving virtual sample generation algorithm is illustrated in Algorithm 2. Lines 2 to 18 constitute the main loop for training the GAN model. Within this loop, lines 5 to 12 represent the training and updating process of the discriminator, while lines 13 to 17 represent the training and updating process of the generator. Lines 10 to 11 are the adaptive noise method. Finally, lines 19 to 22 use the trained generator to generate virtual samples.

C. Real-Virtual Ensemble Training

Although the quality of virtual samples is improved using our adaptive noise approach, it may still be low compared to real samples. They could degrade model accuracy if directly mixed with real samples for training [39]. Therefore, we propose a real-virtual ensemble training algorithm, where two models are trained independently on real and virtual datasets, and aggregated by considering model similarity.

During the t -th round, client i trains a model separately using D_i^R and D^V , obtaining corresponding gradients $g_{i,t}^R$ and $g_{i,t}^V$. Considering the aggregation of two models, we utilize the cosine

Algorithm 3. Real-Virtual Ensemble Training.

Input: Local real dataset D_i^R of client i , and shared virtual dataset D^V .

Output: Gradient of the aggregated model $g_{i,t}$ in the t -th round.

- 1: Train a model using D_i^R , obtaining gradients $g_{i,t}^R$.
- 2: Train a model using D^V , obtaining gradients $g_{i,t}^V$.
- 3: Compute and normalize the cosine similarity $\bar{cs}$.
- 4: Calculate $g_{i,t} = \bar{cs} \times g_{i,t}^R + (1 - \bar{cs}) \times g_{i,t}^V$.

similarity to measure the dissimilarity between the two model parameters. We define the model parameters obtained from training with real dataset as $w_{i,t}^R$, and those from training with aggregated virtual dataset as $w_{i,t}^V$. We compute $cs = \frac{w_{i,t}^R \cdot w_{i,t}^V}{\|w_{i,t}^R\| \cdot \|w_{i,t}^V\|}$, where $\cdot$ represents the dot product of vectors, $\|w_{i,t}^R\|$ and $\|w_{i,t}^V\|$ denote the Euclidean norms of vectors $w_{i,t}^R$ and $w_{i,t}^V$. Since the original cosine similarity cs ranges from -1 to 1 , where negative values indicate opposing gradient directions between real and virtual model parameters, we need to normalize it to $[0, 1]$ range as $\bar{cs} = (cs + 1)/2$ to use as meaningful mixing weights in the gradient aggregation. The linear transformation $\bar{cs} = (cs + 1)/2$ ensures that when models are perfectly aligned ($cs = 1$), we get $\bar{cs} = 1$ giving full weight to real gradients, when completely opposing ($cs = -1$), we get $\bar{cs} = 0$ giving full weight to virtual gradients, and when orthogonal ($cs = 0$), we get $\bar{cs} = 0.5$ providing balanced mixing, which intuitively reflects the similarity-based weighting strategy. Then, we calculate $g_{i,t} = \bar{cs} \times g_{i,t}^R + (1 - \bar{cs}) \times g_{i,t}^V$ to obtain the gradient of the aggregated model. When $\bar{cs}$ approaches 1, it indicates that $w_{i,t}^R$ and $w_{i,t}^V$ are highly similar; otherwise, they differ significantly. Based on this characteristic, the key idea of $g_{i,t}$ is to dynamically adjust the weights of real and virtual dataset in gradient updates according to the similarity between their model parameters. This ensures a balanced contribution of both data types during model training, promoting greater stability and effectiveness. The real-virtual ensemble training algorithm is illustrated in Algorithm 3.

In the non-IID scenario, local real dataset may lack certain classes, while aggregated virtual dataset encompass all classes. This discrepancy in class distribution leads to substantial dissimilarity in model parameters between independently trained models, making aggregation ineffective. To address this issue, we propose a method for real and virtual datasets. Specifically, we augment the local real dataset by incorporating half of the data from the corresponding classes in the aggregated virtual dataset, as shown in Fig. 2. Afterwards the new local real and virtual dataset are utilized to execute Algorithm 3.

D. Adaptive DP-Based Secure Aggregation

To prevent privacy leakage from the gradient updates, DP can be introduced into the FL to protect clients' data privacy [34]. However, it may lead to significant model accuracy loss [40]. Different from the GAN training for noise reduction, we consider another different metric: the gradient clipping factor. An

excessively large value introduces excessive noise, while an overly small value leads to excessive gradient clipping, resulting in information loss. Both situations significantly compromise the model accuracy. Therefore, we design an adaptive DP-based secure aggregation algorithm, which can flexibly adjust the clipping factor while protecting clients' data privacy to mitigate the impact of noise on model accuracy.

Inspired by [57], we first utilize prior knowledge to predict the global gradient of the current round, which is then used as the clipping factor C_t , e.g., $C_t = E[\tilde{g}_{t-1}^2]^{1/2}$, where $E[\tilde{g}_{t-1}^2]$ represents the exponentially weighted average of the squared global gradients from previous rounds. Due to the continuity and incremental nature of the training process, it can be viewed as prior knowledge of the current global gradient [58]. The calculation expression for prior knowledge is given by $E[\tilde{g}_0^2] = 0$ and $E[\tilde{g}_{t-1}^2] = E[\tilde{g}_{t-2}^2] + \gamma \times \{\tilde{g}_{t-1}^2 - E[\tilde{g}_{t-2}^2]\}$, where γ is the hyperparameter.

However, prior knowledge $E[\tilde{g}_0^2] = 0$ in the first round leads to $C_t = E[\tilde{g}_0^2]^{1/2} = 0$. Since $\tilde{g}_{i,t} = \frac{g_{i,t}}{\max(1, \|g_{i,t}\|_2/C_t)} + \mathcal{N}(0, \sigma^2 C_t^2 \mathbf{I})$, 0 cannot be used as a denominator and therefore cannot be used for clipping. Therefore, we introduce a threshold H . When there is insufficient prior knowledge of the results during the early rounds of training (i.e., when $E[\tilde{g}_{t-1}^2]^{1/2} < H$), we set a fixed value C as the clipping factor. When the training is continued until the prior knowledge satisfies $E[\tilde{g}_{t-1}^2]^{1/2} > H$, so that the clipping factor takes $C_t = E[\tilde{g}_{t-1}^2]^{1/2}$. We set $H = e^{-5}$ based on empirical analysis showing that gradient norms typically stabilize within 5–10 training rounds, making this threshold appropriately conservative to ensure smooth transition while avoiding premature activation of the adaptive mechanism. In the t -th round, client i clips $g_{i,t}$, and adds noise, which can be represented as:

$$\tilde{g}_{i,t} = \frac{g_{i,t}}{\max(1, \|g_{i,t}\|_2/C_t)} + \mathcal{N}(0, \sigma^2 C_t^2 \mathbf{I}),$$

$$\text{where } C_t = \begin{cases} C, & E[\tilde{g}_{t-1}^2]^{1/2} < H \\ E[\tilde{g}_{t-1}^2]^{1/2}, & E[\tilde{g}_{t-1}^2]^{1/2} > H \end{cases} \quad (1)$$

where $\mathcal{N}(0, \sigma^2 C_t^2 \mathbf{I})$ represents uses the Gaussian distribution with a mean of 0 and a covariance of $\sigma^2 C_t^2 \mathbf{I}$. As the model continues to converge, the clipping factor C_t decreases as $E[\tilde{g}_{t-1}^2]^{1/2}$ decreases, which will make the noise added to results also becomes smaller.

Model Update: The server receives the noise gradients uploaded by clients and calculates the global gradient $\tilde{g}_t = \sum_{i=1}^N \frac{1}{N} \tilde{g}_{i,t}$. The global gradient is then used to update the global model. Each client downloads the new global model to update its local model and begins a new training round.

IV. THEORETICAL ANALYSIS

We first introduce two definitions and four lemmas and then prove the privacy of GetFed.

Definition 4 (Privacy loss [59]): A randomized algorithm $\mathcal{A}$ satisfies $X \rightarrow O$. For any two neighboring datasets X and X' that differ in only single data, the loss of privacy of $\mathcal{A}$ at output

O is:

$$l_p(\mathcal{A}, X, X', O) \triangleq \log \frac{\Pr[\mathcal{A}(X) = O]}{\Pr[\mathcal{A}(X') = O]}.$$

Definition 5 (Instantaneous statistics [59]): The moment statistic of algorithm $\mathcal{A}$ at moment λ is

$$\alpha(\lambda) \triangleq \max_{X, X'} \log E_{O \sim \mathcal{A}(X)} [\exp(\lambda l_p(\mathcal{A}, X, X', O))].$$

Lemma 1 (Tail bound [59]): For any $\varepsilon > 0$, an algorithm $\mathcal{A}$ satisfies (ε, δ) -differential privacy if the following equation is satisfied:

$$\delta = \min_{\lambda} \exp(\alpha_{\mathcal{A}}(\lambda) - \lambda \varepsilon).$$

Lemma 2 (Composability [59]): If an algorithm $\mathcal{A}$ satisfies differential privacy and f is a random mapping function, then $f \cdot \mathcal{A}$ also satisfies differential privacy.

Lemma 3 (Post-processability [59]): If an algorithm $\mathcal{A}$ is composed of a series of sub-algorithms $\mathcal{A}_1, \dots, \mathcal{A}_T$, then for any moment λ , the upper bound of algorithm $\mathcal{A}$ is the sum of the upper bound of all the sub-algorithms $\mathcal{A}_1, \dots, \mathcal{A}_T$, namely, $\alpha_{\mathcal{A}}(\lambda) \leq \sum_{i=1}^T \alpha_{\mathcal{A}_i}(\lambda)$.

Lemma 4 ([59]): Let f is a mapping function, and $\|f(\cdot)\|_2 \leq 1$. Let $\sigma \geq 1$, and X is a random subset, where each element x_i in X is chosen with probability $q < \frac{1}{16\sigma}$ and is mutually independent. Then for any positive integer $\lambda \leq \sigma^2 \ln \frac{1}{q\sigma}$, the algorithm $\mathcal{A}(X) = \sum_{i \in X} f(x_i) + \mathcal{N}(0, \sigma^2 C^2 \mathbf{I})$ satisfies $\alpha_{\mathcal{A}}(\lambda) \leq \frac{q^2 \lambda (\lambda + 1)}{(1-q)C^2 \sigma^2} + O(\frac{q^3 \lambda^3}{\sigma^3 C^3})$.

GetFed incorporates DP at two critical phases: 1) GAN Training Phase (Algorithm 2): Protects against membership inference attacks on virtual samples; 2) Secure Aggregation Phase: Protects gradient updates during model training. Based on the above content, we present two theorems.

Theorem 1: Privacy-preserving virtual sample generation algorithm (i.e., Algorithm 2) satisfies differential privacy.

Proof: According to Lemma 1, to ensure that Algorithm 2 satisfies differential privacy, it is sufficient to constrain the upper bound of $\alpha_{\mathcal{A}}(\lambda)$. Moreover, the privacy budget (ε, δ) can be further calculated from $\alpha_{\mathcal{A}}(\lambda)$. Let the t -th round of training of the Algorithm 2 be represented as the sub-algorithm $\mathcal{A}_t$, including the training process of the discriminator/generator, denoted as $\mathcal{A}_t^D/\mathcal{A}_t^G$. Step 7 in Algorithm 2 is executed B times, then each gradient is clipped and noise is added, and finally the average gradient is calculated. Therefore, $\mathcal{A}_t^D = \frac{1}{B} (\sum_{b=1}^B g_b^D + \zeta \cdot \mathcal{N}(0, \sigma^2 C^2 \mathbf{I}))$. According to Algorithm 2 and Lemma 4, we can get an approximate upper bound for $\alpha_{\mathcal{A}_t^D}(\lambda)$, which is $\frac{q^2 \zeta \lambda (\lambda + 1)}{(1-q)C^2 \sigma^2 B} + O(\frac{q^3 \lambda^3}{\sigma^3 C^3})$. Combined with Lemma 1, $\mathcal{A}_t^D$ satisfies DP. Since the update of the generator only involves interaction with the discriminator and does not involve real samples, $\mathcal{A}_t^G$ can be viewed as a random mapping applied to the output of $\mathcal{A}_t^D$. According to Lemma 2, training $\mathcal{G}$ does not result in additional privacy loss, namely, $\alpha_{\mathcal{A}_t}(\lambda) = \alpha_{\mathcal{A}_t^G \cdot \mathcal{A}_t^D}(\lambda) = \alpha_{\mathcal{A}_t^D}(\lambda)$.

Algorithm 2 consists of T_{pre} rounds. Combining Lemma 3 and the computed approximate upper bound, we estimate the overall privacy loss. Assume for now that σ, λ satisfy the conditions in Lemma 3. By Lemma 1 and Lemma 4, the log moment of

Algorithm 2 can be bounded as follows $\alpha_A(\lambda) \leq \frac{T_{pre} q^2 \zeta \lambda (\lambda+1)}{(1-q) C^2 \sigma^2 B}$. By Lemma 1 and Lemma 3, to guarantee Algorithm 2 to be (ε, δ) -differential privacy, it suffices that: $\frac{T_{pre} q^2 \zeta \lambda (\lambda+1)}{(1-q) C^2 \sigma^2 B} \leq \lambda \varepsilon / 2$ and $\exp(\lambda \varepsilon / 2) \leq \delta$. In addition, we need $\lambda \leq \sigma^2 \log(1/q\sigma)$. It is now easy to verify that when $\varepsilon = c_1 \frac{q^2 \zeta}{(1-q) C^2 B} T_{pre}$, we can satisfy all these conditions by setting $\sigma = c_2 \frac{q}{\varepsilon C} \sqrt{\frac{\zeta T_{pre} \log(\frac{1}{\delta})}{(1-q) B}}$ for some explicit constants c_1 and c_2 . Therefore, there exist constants c_1 and c_2 such that for any $\varepsilon < c_1 \frac{q^2 \zeta}{(1-q) C^2 B} T_{pre}$, if $\sigma \geq c_2 \frac{q}{\varepsilon C} \sqrt{\frac{\zeta T_{pre} \log(\frac{1}{\delta})}{(1-q) B}}$, then Algorithm 2 satisfies (ε, δ) -differential privacy.

Theorem 2: Adaptive DP-based secure aggregation algorithm satisfies differential privacy.

Proof: According to Lemma 3, the privacy loss of GetFedis related to the number of clients N and the number of training rounds T_{online} . Suppose the total moment statistics of the scheme is $\alpha(\lambda)$ and the moment statistics of client i in round t is $\alpha_i^t(\lambda)$. Since DP is combinable, the following inequalities exist: $\alpha(\lambda) \leq \sum_{t=1}^{T_{online}} \sum_{i=1}^N \alpha_i^t(\lambda)$.

Norm clipping factor guarantees that $\|g_{i,t}\|_2$ is preserved when $\|g_{i,t}\|_2 \leq C_t$. It also guarantees that $\|g_{i,t}\|_2$ scaled down to C_t when $\|g_{i,t}\|_2 > C_t$. After norm clipping, $\alpha_i^t(\lambda)$ is mainly for adding Gaussian noise $N(0, \sigma^2 C_t^2 \mathbf{I})$ to the results. From probability theory, the probability density function of the Gaussian distribution $N(0, \sigma^2 C_t^2 \mathbf{I})$ is $\mu_0(x) = \frac{1}{\sqrt{2\pi\sigma C_t}} e^{-\frac{x^2}{2C_t^2\sigma^2}}$. The probability of the Gaussian distribution $N(1, \sigma^2 C_t^2 \mathbf{I})$ is $\mu_1(x) = \frac{1}{\sqrt{2\pi\sigma C_t}} e^{-\frac{(x-1)^2}{2C_t^2\sigma^2}}$. If we assume that q is the sampling probability, the Gaussian distribution satisfies $\mu = q\mu_0 + (1-q)\mu_1$. In order word, μ is a Gaussian distribution of a mixture of μ_0 and μ_1 .

We obtain $\alpha_i^t(\lambda) = \log \max((\frac{\mu_0(x)}{\mu(x)})^\lambda, (\frac{\mu_1(x)}{\mu(x)})^\lambda)$. Based on the composability and the noise $N(0, \sigma^2 C_t^2 \mathbf{I})$ added locally, we can calculate the total moment statistics $\alpha(\lambda)$. According to Lemma 1, we obtain that GetFedis satisfies $\delta = \exp(\alpha(\lambda) - \lambda\varepsilon)$. Finally, the privacy can be maintained at (ε, δ) -DP for all clients. During the actual operation of the algorithm, the value range of λ at integer time is usually $0 \leq \lambda \leq T_{online}$. $\square$

V. EVALUATION

A. Evaluation Setup

We implement GetFed and the other benchmarks based on the PyTorch framework in Python 3.8 and conduct experiments on a server running Ubuntu 18.04, which is equipped with an Intel(R) Xeon(R) Gold 6133 CPU @ 2.5 GHz, 128 GB RAM, and three NVIDIA 4090 GPUs with 24 GB memory each. The CUDA version is 12.0. The experiments are conducted in a LAN environment. We run all experiments 10 times and calculate the average value as the final experimental results. We illustrate the empirical settings of FL for our experiments in Table I.

Datasets and Models: We conduct experiments on four datasets: MNIST [60], Fashion-MNIST (FMNIST) [61], CIFAR-10 [62], and SVHN [63].

TABLE I
THE DEFAULT PARAMETERS IN OUR EXPERIMENTS

Parameters	Description
$N = 20$	The number of clients.
$\kappa = 0.15$	Sharing ratio factor.
$T_{pre} = 100$	The number of training rounds for GAN.
$T_{online} = 200$	The number of training rounds for FL.
$\mathcal{D}_{lr} = 0.0001$	The learning rate for the discriminator $\mathcal{D}$.
$\mathcal{G}_{lr} = 0.0001$	The learning rate for the generator $\mathcal{G}$.
$(\beta_1, \beta_2) = (0.5, 0.9)$	The Adam optimizer parameters for GAN.
$E = 1$	The number of local epochs.
$B = 128$	Local batch size.
$lr = 0.015$	Learning rate.
momentum = 0.9	SGD momentum.
weight decay = 5e-4	SGD weight decay.
$\mu = 1$	The regularization parameter.
$H = e-5$	The threshold in DP.
$C = 2$	DP model clip parameter.
$\sigma = 0.15$	DP Gauss noise parameter.
$(\varepsilon, \delta) = (100, 0.0001)$	Privacy budget in DP.

- MNIST contains 10 classes with a total of 60 000 training images and 10 000 testing images. Each is a grayscale image of size 28×28 pixels.
- FMNIST consists of 60 000 grayscale training images and 10 000 grayscale testing images, all sized 28×28 .
- CIFAR-10 contains 10 classes with a total of 50 000 training images and 10 000 testing images. Each is a color image of size 32×32 pixels.
- SVHN contains about 100 000 images, which are divided into two sets: a training set with 73,257 images and a testing set with 26 032 images.

We employ multiple deep neural networks to train in GetFed, including a LeNet-5 model [64] on MNIST and Fashion-MNIST, a GoogleNet model [65] on CIFAR-10, and a ResNet-18 model [66] on SVHN.

Data Distributions: To simulate the non-IID, we introduce two data distributions, such as the Pathological distribution $Path(\tau)$ [1] and the Dirichlet distribution $Dir(\iota, p)$ [67].

Pathological Distribution: In this setting, each client i receives data from exactly τ classes out of the total U classes. The data partitioning process works as follows: (1) Sort all training samples by their labels; (2) Divide the sorted data into $N \times \tau$ shards, where each shard contains samples from only one class; (3) Randomly assign τ shards to each client, ensuring that each client receives data from exactly τ distinct classes. This creates a highly skewed distribution where clients have access to only a subset of classes.

Dirichlet Distribution: Each client i samples a probability vector from $Dir(\iota, p)$, where ι controls the concentration parameter and p is the prior distribution over labels (typically uniform). The data allocation process is: (1) For each class k , determine the proportion of samples that client i receives; (2) Allocate the corresponding number of samples from class k to client i based on this proportion. A smaller ι value results in more concentrated distributions, leading to higher degrees of statistical heterogeneity among clients.

We use the pathological distribution $\tau \in \{2, 5\}$ and the Dirichlet distribution $\iota \in \{0.1, 1.0\}$. Where $\tau = 2$ and $\iota = 0.1$

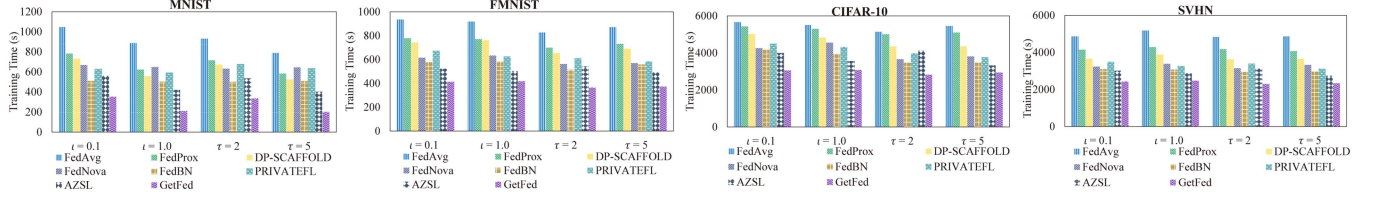


Fig. 3. The training time of different schemes in various non-IID settings for multiple models.

represent the extreme non-IID situation, and $\tau = 5$ and $\iota = 1.0$ represent the mild non-IID situation. These settings ensure a comprehensive evaluation on diverse non-IID data.

Baselines: To validate the proposed GetFed, we introduce the following FL schemes for comparison. FedAvg [1] is a well-established and classic FL method, serving as the benchmark for results. FedProx [12] utilizes a regularization parameter μ to mitigate the model divergence and adds noise to gradient updates by leveraging DP. DP-SCAFFOLD [13] incorporates DP constraints into the popular SCAFFOLD [68] algorithm. FedNova [17] employs a normalization-averaging approach to mitigate target inconsistency and address non-IID issues. FedBN [18] uses local batch normalization to alleviate the non-IID problem before averaging models. PRIVATEFL [16] is to jointly learn a differentially private, personalized data transformation for each client during local training. AZSL [69] utilizes the Wasserstein Generative Adversarial Network with Gradient Penalty for generating virtual data.

B. Performance Evaluation on Model Accuracy

We evaluate the global model accuracy on different datasets in different data distributions. Additionally, We conduct the model accuracy of centralized training as an upper bound. For the comparison schemes, we set the default privacy budget (ϵ, δ) to (100, 0.0001), as shown in Table I. Table II shows the results and we have the following two valuable findings.

First, as the different distributions among clients ι or the different distributions among clients τ decrease, the testing accuracy of all schemes decreases. However, GetFed still achieves the highest model accuracy. For example, with $\iota = 0.1$, GetFed outperforms others by 46.58% (FedAvg), 32.68% (FedProx), 20.97% (DP-SCAFFOLD), 14.53% (FedNova), 6.17% (FedBN), 2.98% (PRIVATEFL), and 2.61% (AZSL) on CIFAR-10. With $\tau = 2$, GetFed surpasses others by 30.94% (FedAvg), 28.84% (FedProx), 12.08% (DP-SCAFFOLD), 9.44% (FedNova), 5.29% (FedBN), 3.63% (PRIVATEFL), and 5.34% (AZSL) on CIFAR-10. The results indicate that the diversity of classes in clients' local data significantly affects model accuracy. Our scheme ensures that each client has samples from all classes, which is why GetFed achieves higher model accuracy.

Second, GetFed exhibits increasingly superior performance as the model complexity rises. MNIST and FMNIST are relatively simple tasks where the image has only one channel and the simple LeNet-5 model can accomplish these tasks. In contrast, CIFAR-10 and SVHN are complex tasks where the image has three channels and we need to use complex models such as

TABLE II
THE TESTING ACCURACY (%) RESULTS OF DIFFERENT SCHEMES

Dataset	Scheme	$\iota = 0.1$	$\iota = 1.0$	$\tau = 2$	$\tau = 5$
MNIST	FedAvg	79.81	81.35	80.99	82.18
	FedProx	77.88	78.87	84.39	89.42
	DP-SCA	72.56	76.34	86.35	91.29
	FedNova	83.13	86.11	90.25	93.46
	FedBN	87.67	90.85	92.76	95.23
	PRIVATEFL	93.25	95.96	94.83	96.36
	AZSL	95.23	96.26	95.41	97.12
	GetFed	98.29	98.85	97.86	98.55
	Centralized	99.21			
FMNIST	FedAvg	48.37	72.55	42.33	68.37
	FedProx	70.23	76.23	53.47	63.24
	DP-SCA	63.91	80.25	60.93	69.47
	FedNova	67.83	77.93	63.74	71.26
	FedBN	73.16	79.46	68.19	73.46
	PRIVATEFL	72.95	81.36	73.94	77.69
	AZSL	75.16	83.02	74.26	77.26
	GetFed	76.34	85.53	77.34	80.91
	Centralized	91.90			
CIFAR-10	FedAvg	24.32	40.97	33.63	48.97
	FedProx	38.22	43.58	35.73	51.78
	DP-SCA	49.93	55.37	52.49	63.93
	FedNova	56.37	62.19	55.13	66.76
	FedBN	64.73	77.36	59.28	70.23
	PRIVATEFL	67.92	80.46	60.94	75.36
	AZSL	68.29	79.16	59.23	76.42
	GetFed	70.90	84.35	64.57	79.81
	Centralized	92.62			
SVHN	FedAvg	33.83	50.48	48.56	59.32
	FedProx	42.78	55.67	59.69	67.36
	DP-SCA	51.34	65.62	62.93	74.90
	FedNova	63.84	70.93	65.36	79.32
	FedBN	76.68	81.36	68.73	83.47
	PRIVATEFL	82.36	87.69	73.36	87.64
	AZSL	84.26	90.16	75.42	88.43
	GetFed	87.81	92.68	77.09	90.52
	Centralized	95.57			

DP-SCA refers to DP-SCAFFOLD. We take the centralized training results as an upper bound on the accuracy. GetFed is the best in all situations.

GoogleNet and ResNet-18. Particularly on complex models, GetFed achieves a greater model accuracy improvement across all settings (approaching the upper bound), suggesting that GetFed is more applicable in real-world scenarios.

C. Performance Evaluation on Training Time

In FL, the training time directly impacts the availability. We conduct a series of tests on various models to evaluate the

TABLE III
GETFEDINVOLVES THE PRE-PROCESSING TIME (S) ON DIFFERENT DATASETS

Dataset	$\iota = 0.1$	$\iota = 1.0$	$\tau = 2$	$\tau = 5$
MNIST	130.13	138.95	115.78	119.54
FMNIST	136.04	149.63	123.93	129.64
CIFAR-10	272.08	273.47	263.68	266.59
SVHN	253.53	258.24	247.37	242.35

training time, and the results are shown in the Fig. 3. The training time primarily validates the following two key points:

- *Online Training Time:* Is the training time of GetFed better than other six schemes?
- *Overall Training Time:* Considering the time spent in the pre-processing phase, is the overall training time of GetFed still within an acceptable range?

GetFed is faster than other six schemes since it has the shortest online training time until the model converges in various non-IID settings. For example, with $\iota = 0.1$, GetFed surpasses others by $1.88\times$ (FedAvg), $1.78\times$ (FedProx), $1.65\times$ (DP-SCAFFOLD), $1.40\times$ (FedNova), $1.38\times$ (FedBN), $1.48\times$ (PRIVATEFL), and $1.35\times$ (AZSL) on CIFAR-10. With $\tau = 2$, GetFed surpasses others by $1.82\times$ (FedAvg), $1.77\times$ (FedProx), $1.54\times$ (DP-SCAFFOLD), $1.29\times$ (FedNova), $1.23\times$ (FedBN), $1.28\times$ (PRIVATEFL), and $1.26\times$ (AZSL) on CIFAR-10. Because FL is a synchronous system, the online training time per round depends on the slowest-performing client. Each scheme has a relatively small difference about online training time in different distribution scenarios.

To enhance model accuracy in non-IID settings, FedAvg requires more rounds of iterative training. FedProx and DP-SCAFFOLD incur high computation costs and require more rounds by tuning the weight parameters or making an additional pass over the previously computed gradients. FedNova and FedBN introduce extra computation overhead due to the employment of batch normalization. PRIVATEFL relies on data transformations that result in more iterations for convergence. AZSL requires more rounds to train the GAN network. Compared with these schemes, GetFed achieves the lowest training time by addressing the non-IID issue more effectively, enabling faster model convergence.

We also need to take into account the pre-processing phase of GetFed. The pre-processing phase primarily involves training SNGAN and generating virtual samples. Based on the results in Table III and Fig. 3, it can be observed that the time spent in the pre-processing phase of GetFed is approximately 8.92% to 32.95% of the training time.

Next, we consider whether the client is capable of training SNGAN and sustaining the communication overhead of uploading virtual samples. We conducted a thorough analysis of these additional costs compared to FedAvg. The results are shown in Table IV.

FLOPS (Floating-point Operations Per Second) is a standard metric for measuring computation performance. We employ it to compare the training costs of GoogleNet and SNGAN on CIFAR-10. The left half of Table IV shows that the

TABLE IV
THE COMPARISON OF ADDITIONAL COMPUTATION AND COMMUNICATION COSTS

FLOPS (G)		Size (MB)	
GoogleNet	SNGAN	Parameters	Samples
90.32	84.89	44.86	21.35

computation cost of GoogleNet and SNGAN is similar, which means clients have the ability to train SNGAN. Additionally, the trained SNGAN can be reused for the same dataset. In the future, the training cost of SNGAN would be zero.

The extra communication cost introduced by SNGAN is the virtual samples that are transmitted. The communication cost of FedAvg is the parameters of the local model. As shown in the right half of Table IV, the extra communication cost is much smaller. Local model parameters need to be uploaded in every communication round, whereas virtual samples are uploaded only once in the pre-processing phase. Therefore, as long as a device can complete FedAvg, GetFed can be widely adopted in computing applications.

D. Impact of Privacy Parameters

Based on different datasets and data distributions, experiments are conducted using SNGAN trained under five different settings: $\varepsilon = 1, 10, 20, 50, 100$, and ∞ (where ∞ indicates training SNGAN without adding noise). The testing accuracy of the global model and the membership inference attack [45] in GetFed are evaluated. Specifically, we train an attack model to conduct the membership inference attack on data generated by SNGAN with different privacy settings. The attack model is pretrained to predict sample membership from a static given dataset, which consists of 10% of samples from the SNGAN training set and an additional 90% of unrelated samples. The samples are either from the SNGAN training set or unrelated samples, with the attack model predicting them correctly, considered as successful attacks (i.e., TP and TN). Any other classification scenario, such as a sample from the SNGAN training set predicted as unrelated or a sample from unrelated sources predicted as from the SNGAN training set, is considered an attack failure (i.e., FP and FN). Therefore, the attack success rate is $(TP + TN)/(TP + TN + FP + FN)$. Similar to previous studies [45], [70], a lower attack success rate indicates poorer model prediction ability, with values typically below 40% suggesting that the attack model lacks sufficient prediction ability. The results are presented in Table V. Fig. 4 illustrates the virtual samples generated by SNGAN under different privacy budgets.

As the parameter ε increases, the testing accuracy improves in four data distributions and the membership inference attack accuracy rises. When no noise is added ($\varepsilon = \infty$), the testing accuracy is highest, and the accuracy of membership inference attack achieves about 40%. Then as ε decreases, the membership inference attack accuracy drops below 20%, which reflects the efficiency of adding noises in SNGAN. When the privacy budget is within the range of 20 to 100, the testing accuracy is relatively

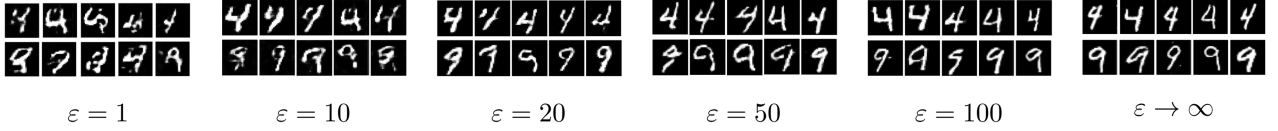
Fig. 4. Virtual samples generated under different privacy budgets ϵ . We use digits 4 and 9 in MNIST as representative examples.

TABLE V
THE TESTING ACCURACY (%) / THE MEMBERSHIP INFERENCE ATTACK
ACCURACY (%) RESULTS OF DIFFERENT PRIVACY PARAMETERS IN VIRTUAL
SAMPLE GENERATION

	ϵ	$\iota = 0.1$	$\iota = 1.0$	$\tau = 2$	$\tau = 5$
MNIST	1	82.13/ 9.26	83.46/ 8.46	80.16/ 10.26	81.98/ 10.49
	10	90.19/10.39	91.67/11.97	90.46/10.02	90.99/12.49
	20	96.31/14.39	96.88/15.77	95.78/13.55	96.15/16.61
	50	97.12/24.96	97.63/23.64	96.89/22.97	97.02/25.74
	100	98.29/36.46	98.85/35.97	97.86/37.92	98.55/35.91
	∞	98.54 /45.36	98.97 /44.99	98.64 /42.31	99.02 /44.36
FMNIST	1	50.19/ 10.23	52.64/ 11.24	49.36/ 12.16	53.97/ 9.46
	10	62.41/12.37	67.38/12.97	60.17/13.69	63.93/11.39
	20	75.07/16.39	82.99/15.97	76.12/17.06	77.83/17.39
	50	75.36/25.36	83.29/26.39	76.93/24.19	78.12/26.26
	100	76.34/34.36	85.53/33.79	77.34/32.23	80.91/35.51
	∞	82.16 /42.36	88.15 /43.62	81.23 /44.62	86.93 /45.92
CIFAR-10	1	45.26/ 8.32	52.19/ 7.42	33.26/ 9.26	47.96/ 7.02
	10	58.93/9.86	64.37/9.76	46.73/10.26	55.95/8.94
	20	67.18/12.98	82.38/13.75	61.37/11.48	76.93/13.92
	50	69.27/18.61	83.16/17.71	62.18/16.79	78.07/19.34
	100	70.90/25.93	84.35/26.97	64.57/27.93	79.81/24.41
	∞	77.34 /33.68	88.89 /34.39	70.37 /34.98	84.97 /35.23
SVHN	1	60.26/ 9.26	65.19/ 10.23	58.72/ 10.29	63.19/ 9.37
	10	74.63/10.68	76.91/12.92	65.67/13.32	74.17/11.13
	20	84.36/14.39	85.03/16.36	71.36/15.97	83.91/13.14
	50	86.52/19.63	88.68/19.88	74.46/18.99	86.16/17.66
	100	87.81/27.69	92.68/25.79	77.09/24.34	90.52/28.92
	∞	89.36 /35.16	93.74 /36.98	82.36 /34.19	92.46 /36.94

TABLE VI
THE TESTING ACCURACY (%) / MEMBERSHIP INFERENCE ATTACK ACCURACY

		$\iota = 0.1$	$\iota = 1.0$	$\tau = 2$	$\tau = 5$
①	w.	97.12 /24.96	97.63 /23.64	96.89 /22.97	97.02 /25.74
	w/o.	94.36/ 22.95	95.02/ 22.45	93.65/ 20.78	94.83/ 23.97
②	w.	75.36 /25.36	83.29 /26.39	76.93 /24.19	78.12 /26.26
	w/o.	72.36/ 23.69	78.92/ 25.16	70.83/ 21.39	71.94/ 25.36
③	w.	69.27 /18.61	83.16 /17.71	62.18 /16.79	78.07 /19.34
	w/o.	58.69/ 15.97	75.94/ 15.78	50.97/ 14.68	64.69/ 18.39
④	w.	86.52 /19.63	88.68 /19.88	74.46 /18.99	86.16 /17.66
	w/o.	71.69/ 18.74	74.51/ 17.79	63.49/ 15.97	72.39/ 14.97

(%) results of whether (w.) or not (w/o.) the adaptive method is used in virtual sample generation. ① denotes MNIST. ② denotes FMNIST. ③ denotes CIFAR-10. ④ Denotes SVHN.

close to each other. However, when $\epsilon = 10$, the testing accuracy drops significantly. As the privacy budget decreases, more noise is added, resulting in lower image quality, which impacts model training in FL (e.g., $\epsilon = 1$). This highlights the trade-off between the usability and privacy of SNGAN.

Table VI presents the results of the testing accuracy and membership inference attack accuracy when employing adaptive method and the original DP method. The membership

TABLE VII
COMPARISON WITH STATE-OF-THE-ART DP-GAN METHODS ($\epsilon = 100$)

Dataset	Non-IID	Method	FID ($\downarrow$)	Privacy Budget ($\downarrow$)
MNIST	$\iota = 0.1$	F	13.82	2.53
		D	8.93	1.81
	$\iota = 1.0$	F	12.95	2.42
		D	8.25	1.67
	$\tau = 2$	F	14.77	2.67
		D	9.41	1.96
	$\tau = 5$	F	14.15	2.58
		D	8.70	1.73
CIFAR-10	$\iota = 0.1$	F	42.12	10.52
		D	21.88	7.46
	$\iota = 1.0$	F	35.97	8.33
		D	16.42	6.93
	$\tau = 2$	F	45.63	9.82
		D	24.27	7.92
	$\tau = 5$	F	38.37	9.41
		D	18.12	7.38

“F” denotes traditional fixed-noise DP GAN training; “D” presents our adaptive noise reduction GAN training.

inference attack accuracy calculation method is the same as that in Table V. Using the adaptive method, the membership inference attack accuracy remains comparable, while the testing accuracy improves by 3–14%. This indicates that employing the adaptive method effectively mitigates the adverse impact of noise on model accuracy while ensuring effective protection of the real training samples’ privacy.

Table VII presents a comprehensive comparison of our adaptive noise reduction mechanism with traditional fixed-noise DP GAN training approach across two datasets and four non-IID distributions. Our adaptive method consistently outperforms the fixed-noise baseline across all experimental settings, achieving 35–54% better FID scores and 16–32% privacy budget reduction.

E. Impact of Sharing Ratio Factors of Virtual Samples

We test the impact of different sharing ratio factors of virtual samples κ on model accuracy. κ is set to 0.01, 0.05, 0.15, 0.30, and 0.50 for testing the model accuracy with respective data distributions on four different datasets, and the results are presented in Table VIII.

Increasing the sharing ratio factor of virtual samples helps balance the data distribution and improve model accuracy. For the MNIST, as the distribution differences (i.e., ι and τ increase) decrease, the change in sharing ratio factor has little impact on model accuracy, and different values can yield similar results.

TABLE VIII
THE TESTING ACCURACY (%) RESULTS OF DIFFERENT SHARING RATIO
FACTORS OF VIRTUAL SAMPLES κ

Dataset	κ	$\iota = 0.1$	$\iota = 1.0$	$\tau = 2$	$\tau = 5$
MNIST	0.01	94.93	95.63	94.67	95.62
	0.05	96.32	97.02	95.64	96.79
	0.15	98.29	98.85	97.86	98.55
	0.30	97.46	97.39	96.32	96.97
	0.50	96.13	96.78	95.36	95.47
FMNIST	0.01	70.85	85.19	69.51	74.26
	0.05	72.34	85.13	73.58	75.12
	0.15	76.34	85.53	77.34	80.91
	0.30	75.25	81.78	76.51	78.41
	0.50	73.23	77.36	73.61	76.49
CIFAR-10	0.01	62.67	75.32	58.59	72.76
	0.05	64.97	77.67	59.37	74.76
	0.15	70.90	84.34	64.57	79.81
	0.30	67.67	80.16	62.76	77.03
	0.50	64.23	72.96	59.39	68.79
SVHN	0.01	78.63	80.63	67.36	80.46
	0.05	83.68	85.29	71.92	84.92
	0.15	87.81	92.68	77.09	90.52
	0.30	84.61	88.99	73.48	82.74
	0.50	79.63	81.36	70.94	75.34

However, like CIFAR-10 and SVHN, models with higher sharing ratio factors perform worse than those with lower sharing ratio factors. This is because the introduction of noise during SNGAN training results in lower-quality virtual samples. While increasing the number of virtual samples better balances the data distribution, it degrades the overall sample quality, creating a trade-off between data distribution and sample quality. Therefore, it's essential to strike a balance between the sharing ratio factor of virtual samples and model accuracy when selecting an appropriate κ .

F. Effectiveness of Adaptive DP-Based Secure Aggregation

To measure the effectiveness of adaptive DP-based secure aggregation in reducing privacy budget consumption, we test the privacy budget consumed at the end of model training and compared it with PRIVATEFL. Specifically, we set the clipping parameter C to 2 and record the privacy budget consumed by both schemes under different datasets and data distributions when completing the FL task. We also evaluate the model accuracy in different thresholds of adaptive DP-based secure aggregation, and compare it with traditional DP. The results are shown in Table IX and Table X.

GetFed significantly reduces the consumption of privacy budget, with reductions ranging from 7.98% to 32.26%. This demonstrates the effective reduction of privacy budget consumption by our adaptive DP-based secure aggregation in various non-IID scenarios. Particularly, on CIFAR-10 and SVHN, GetFed exhibits a more pronounced improvement in privacy budget savings compared to PRIVATEFL. The testing accuracy of using dynamic differential privacy is higher than traditional differential privacy. Moreover, to achieve the best testing accuracy, we need to set an appropriate threshold H .

TABLE IX
ADAPTIVE DP-BASED SECURE AGGREGATION ON PRIVACY BUDGET
CONSUMPTION

Dataset	non-IID	PFL	GetFed	Reduction	Avg
MNIST	$\iota = 0.1$	2.03	1.81	10.83%	11.67%
	$\iota = 1.0$	1.94	1.67	13.92%	
	$\tau = 2$	2.13	1.96	7.98%	
	$\tau = 5$	2.01	1.73	13.93%	
FMNIST	$\iota = 0.1$	2.59	2.09	19.31%	24.97%
	$\iota = 1.0$	2.32	1.83	21.12%	
	$\tau = 2$	3.41	2.31	32.26%	
	$\tau = 5$	2.65	1.93	27.17%	
CIFAR-10	$\iota = 0.1$	9.80	7.46	23.88%	27.02%
	$\iota = 1.0$	9.57	6.93	27.59%	
	$\tau = 2$	11.37	7.92	30.34%	
	$\tau = 5$	10.01	7.38	26.27%	
SVHN	$\iota = 0.1$	12.39	10.13	18.24%	18.17%
	$\iota = 1.0$	11.03	9.78	11.33%	
	$\tau = 2$	13.94	10.82	22.38%	
	$\tau = 5$	12.97	10.28	20.74%	

PFL refers to PRIVATEFL.

TABLE X
THE THRESHOLD OF ADAPTIVE DP-BASED SECURE AGGREGATION AFFECTS
TESTING ACCURACY (%)

Dataset	non-IID	$H=e-3$	$H=e-5$	$H=e-8$	No.
MNIST	$\iota = 0.1$	96.37	98.29	94.36	92.19
	$\iota = 1.0$	97.02	98.85	95.43	94.02
	$\tau = 2$	95.74	97.86	96.36	93.84
	$\tau = 5$	96.03	98.55	97.42	95.33
CIFAR-10	$\iota = 0.1$	67.25	70.90	68.29	63.72
	$\iota = 1.0$	78.36	84.35	80.26	74.39
	$\tau = 2$	59.32	64.57	58.73	50.62
	$\tau = 5$	70.86	79.81	73.64	67.51

“No.” denotes traditional differential privacy.

G. Effectiveness of Real-Virtual Ensemble Training

To demonstrate how our real-virtual ensemble training algorithm can strike a good trade-off between virtual and real samples, we first evaluate the effectiveness of our proposed the optimization method on real and virtual datasets. “Plain” means that virtual samples and real samples are mixed for training without any optimization. “Optimized” means using our proposed real-virtual ensemble training solution. Then, we conduct ablation experiments where virtual and real samples are trained individually as well as virtual and real samples are trained separately or aggregated together. The results are shown in Table XI and Fig. 5. In Fig. 5, “Real sample training” represents training only with real samples, “Virtual sample training” represents training only with virtual samples, “Aggregation training” represents training with two types of samples simultaneously, and “Separate training” represents training with the two types of samples separately based on our proposed the optimization method.

The optimization method on real and virtual datasets consistently achieve the highest model accuracy compared to the directly separate plain method. With this optimization, the two models are trained on nearly the same data distribution, which

TABLE XI
PLAIN VS. OPTIMIZED TESTING ACCURACY (%) RESULTS FOR LOCAL REAL
DATASET IN THE REAL-VIRTUAL ENSEMBLE TRAINING ALGORITHM

Dataset	Method	$\iota = 0.1$	$\iota = 1.0$	$\tau = 2$	$\tau = 5$
MNIST	Plain	95.36	96.34	93.48	92.67
	Optimized	98.29	98.85	97.86	98.55
FMNIST	Plain	72.51	77.89	72.35	76.88
	Optimized	76.34	85.53	77.34	80.91
CIFAR-10	Plain	64.26	75.89	61.84	73.69
	Optimized	70.90	84.35	64.57	79.81
SVHN	Plain	80.54	86.25	71.26	84.63
	Optimized	87.81	92.86	77.09	90.52

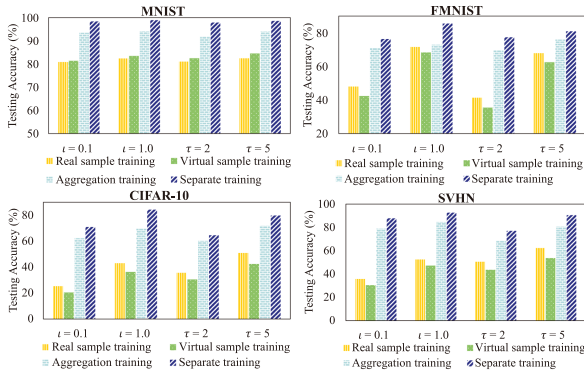


Fig. 5. The testing accuracy in different non-IID settings on multiple datasets.

avoids excessive differences in the parameters of the models obtained from training, and thus better aggregates the two models on the client side.

We observe that training virtual samples and real samples separately and then applying the aggregation algorithm consistently resulted in the highest model accuracy compared to other training methods. Trained only with real samples, the non-IID problem persists. Training only with virtual samples, although the non-IID problem does not exist, the model accuracy remains low due to the lower quality of virtual samples. Directly aggregating training may lead to an overall decrease in sample quality. By using our real-virtual ensemble training algorithm, we can strike a balance between the diversity introduced by virtual samples and the authenticity offered by real samples to improve global model accuracy.

H. Discussion

Our experimental results demonstrate that GetFed improves the model accuracy and convergence speed in the non-IID scenario. An obvious implementation issue then is the choice of sharing ratio factor κ since it dictates the trade-off between the communication overhead and model accuracy. Without prior experience, determining the most appropriate κ in FL can be time-consuming. A simple way to address this problem is to run a few rounds on a few clients to test the suitability of κ . However, this approach lacks a strong theoretical foundation and can face challenges in accurately identifying the optimal κ in FL. One possible direction is to design better GANs to

reduce the quality gap between virtual and real samples. This may only require a smaller κ , which can balance the data distribution without reducing the accuracy. Another direction is to reduce the virtual data that needs to be shared, which requires identifying the features of the virtual data that are most important for solving non-IID problems, thereby reducing the communication overhead without increasing κ . Additionally, future work will focus on developing adaptive mechanisms that can automatically determine the optimal κ value by leveraging real-time feedback from model convergence, data heterogeneity, and virtual sample quality indicators to minimize the trade-off between communication and accuracy.

VI. RELATED WORK

FL on Non-IID Data: FL suffers from the issue that data is usually non-independent and identically distributed. There are many schemes proposed to improve the FL mechanism to solve the non-IID problem, which can be divided into two main categories: model-centric FL and data-centric FL.

Model-centric FL refers to the process of unifying and coordinating the training of local models by adding or modifying loss functions or using global knowledge. Therefore, it can be further divided into two sub-directions: regularization of local training loss and improvement of central server aggregation. First, adding regularization terms to the local training loss in order to control and mitigate local model bias. FedProx [12] incorporated the weighted L_2 regularization in the local loss function to minimize the gap between local models and the global model. DP-SCAFFOLD [13] utilized control variates to steer client-side updates in a specific direction, mitigating the impact of non-IID data distribution. PRIVATEFL [16] was to jointly learn a differentially private, personalized data transformation for each client during local training. Second, improving the aggregation method at the central server to make the aggregated model updates align better with the global optimization direction. FedNova [17] employed a normalization-averaging approach to mitigate target inconsistency, all while maintaining rapid error convergence. FedBN [18] used local batch normalization to alleviate the non-IID problem before averaging models. These two categories of methods offer the advantage of modularity and ease of implementation. However, they are unable to address the essence of the non-IID problem, due to the fundamental distinction between minimizing local empirical loss and minimizing global empirical loss.

In contrast, data-centric FL primarily concentrates on data-level processing to generate more balanced datasets, consequently enhancing the accuracy of the global model. Prior approaches involved various techniques such as data synthesis [25], [71], [72] and data sampling [73], [74]. [25] shared virtual data generated with GAN to avoid sharing raw data, but did not analyse some problems caused by GAN. FedMix [74] proposed a data-averaging method that averages multiple data and allows clients to modify their local data distribution by sharing these averaged data. [71] proposed a zero-time data augmentation method, where the client can generate virtual data based on the global model parameters of the previous

round without having to access the real data of other clients. FLIGAN [72] incorporated techniques such as classwise sampling and node grouping, which were designed to improve the performance of GAN in FL with non-IID settings. CAP [75] proposed a Mix-Generator module which divided a generator into the sharing layer and personalizing layer to address non-IID issues. However, these schemes lack rigorous security proof, raising concerns about potential privacy leakage.

Privacy-Preserving FL: The privacy protection of FL is not as strong as expected. A common finding is that private information about the raw data can be extracted from the intermediate results uploaded by each client in FL [30], [31], [32]. In addition, in data-centric FL, there may be privacy leakage because part of the data is to be shared [25], [28]. For example, a potential privacy problem is that membership inference attacks can still be applied to virtual data to infer whether individual data belongs to the training dataset. However, the use of cryptographic techniques leads to significant computation and/or communication overheads [76], [77], [78], [79]. DP is a lightweight privacy-preserving measure [33], [34]. However, DP might result in a serious loss of model accuracy [16], [36], [37], so we propose GetFedto achieve a balance between privacy and accuracy effectively.

VII. CONCLUSION

In this paper, we present GetFed, an accurate and differentially private FL framework with GAN-based Data Generation on non-IID Data. Specifically, we design privacy-preserving virtual sample generation and DP-based secure aggregation, which adjust the added noise, achieving the balance between accuracy and privacy. Furthermore, we implement a two-layer model aggregation paradigm, effectively mixing virtual and real samples, where two models are trained independently on real and virtual datasets, and aggregated by considering model similarity. Comprehensive experiments demonstrate the advantages of GetFedin achieving higher model accuracy and lower training time. In future work, we will further investigate the trade-off between DP utility and privacy to improve the effectiveness of GetFedwhen dealing with more complex datasets (e.g., ImageNet) in image field. For example, deploy more advanced DP variant methods within our framework to further enhance model accuracy and training efficiency, while providing stronger privacy protection. We will also extend GetFedto real-world applications in critical domains like IoT security [80], where non-IID data is prevalent.

REFERENCES

- [1] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. 20th Int. Conf. Artif. Intell. Statist.*, 2017, pp. 1273–1282.
- [2] E. Khamtsova, C. Hammerschmidt, S. Lagraa, and R. State, "Federated learning for cyber security: SOC collaboration for malicious URL detection," in *Proc. IEEE 40th Int. Conf. Distrib. Comput. Syst.*, 2020, pp. 1316–1321.
- [3] V. Mothukuri, P. Khare, R. M. Parizi, S. Pouriyeh, A. Dehghantanha, and G. Srivastava, "Federated-learning-based anomaly detection for IoT security attacks," *IEEE Internet Things J.*, vol. 9, no. 4, pp. 2545–2554, Feb. 2022.
- [4] H. Fereidooni et al., "FedCRI: Federated mobile cyber-risk intelligence," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2022.
- [5] I. Dayan et al., "Federated learning for predicting clinical outcomes in patients with COVID-19," *Nature Med.*, vol. 27, pp. 1735–1743, 2021.
- [6] Z. Liu et al., "Contribution-aware federated learning for smart healthcare," in *Proc. AAAI Conf. Artif. Intell.*, 2022, pp. 12396–12404.
- [7] D. C. Nguyen et al., "Federated learning for industrial Internet of Things in future industries," *IEEE Wireless Commun.*, vol. 28, no. 6, pp. 192–199, Dec. 2021.
- [8] M. Mehta and C. Shao, "Federated learning-based semantic segmentation for pixel-wise defect detection in additive manufacturing," *J. Manuf. Syst.*, vol. 64, pp. 197–210, 2022.
- [9] Y. Zhao, M. Li, L. Lai, N. Suda, D. Civin, and V. Chandra, "Federated learning with non-IID data," 2018, *arXiv:1806.00582*.
- [10] K. Hsieh, A. Phanishayee, O. Mutlu, and P. Gibbons, "The non-IID data quagmire of decentralized machine learning," in *Proc. 37th Int. Conf. Mach. Learn.*, 2020, pp. 4387–4398.
- [11] P. Kairouz et al., "Advances and open problems in federated learning," *Foundations Trends Mach. Learn.*, vol. 14, pp. 1–210, 2021.
- [12] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," in *Proc. Mach. Learn. Syst.*, 2020, pp. 429–450.
- [13] M. Noble, A. Bellet, and A. Dieuleveut, "Differentially private federated learning on heterogeneous data," in *Proc. Int. Conf. Artif. Intell. Statist.*, 2022, pp. 143–150.
- [14] T.-M. H. Hsu, H. Qi, and M. Brown, "Federated visual classification with real-world data distribution," in *Proc. Computer Vis.–ECCV 2020: 16th Eur. Conf. Glasgow, U.K.*, 2020, pp. 76–92.
- [15] D. A. E. Acar, Y. Zhao, R. M. Navarro, M. Mattina, P. N. Whatmough, and V. Saligrama, "Federated learning based on dynamic regularization," in *Proc. Int. Conf. Learn. Representations*, May 2021.
- [16] Y. Yang, B. Hui, H. Yuan, N. Gong, and Y. Cao, "{PrivateFL}: Accurate, differentially private federated learning via personalized data transformation," in *Proc. 32nd USENIX Secur. Symp.*, 2023, pp. 1595–1612.
- [17] J. Wang, Q. Liu, H. Liang, G. Joshi, and H. V. Poor, "Tackling the objective inconsistency problem in heterogeneous federated optimization," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 7611–7623.
- [18] X. Li, M. Jiang, X. Zhang, M. Kamp, and Q. Dou, "Fedbn: Federated learning on non-IID features via local batch normalization," in *Proc. Int. Conf. Learn. Representations*, May 2021.
- [19] T.-M. H. Hsu, H. Qi, and M. Brown, "Measuring the effects of non-identical data distribution for federated visual classification," in *Proc. NeurIPS Workshop Federated Learn.*, Vancouver, Canada, Dec. 2019.
- [20] C. He, M. Annamalai, and S. Avestimehr, "Group knowledge transfer: Federated learning of large CNNs at the edge," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 14068–14080.
- [21] Z. Zhu, J. Hong, and J. Zhou, "Data-free knowledge distillation for heterogeneous federated learning," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 12878–12889.
- [22] J. Shao, Y. Sun, S. Li, and J. Zhang, "Dres-FL: Dropout-resilient secure federated learning for non-IID clients via secret data sharing," *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 10533–10545.
- [23] E. Jeong, S. Oh, H. Kim, J. Park, M. Bennis, and S.-L. Kim, "Communication-efficient on-device machine learning: Federated distillation and augmentation under non-IID private data," in *Proc. NIPS Workshop Mach. Learn. Phone Other Consum. Devices (MLPCD 2)*, Montréal, Canada, Dec. 2018.
- [24] J. Goetz and A. Tewari, "Federated learning via synthetic data," 2020, *arXiv:2008.04489*.
- [25] Z. Li, J. Shao, Y. Mao, J. H. Wang, and J. Zhang, "Federated learning with GAN-based data synthesis for non-IID clients," in *Proc. Int. Workshop Trustworthy Federated Learn.*, 2022, pp. 17–22.
- [26] I. Goodfellow et al., "Generative adversarial nets," in *Proc. Adv. Neural Inf. Process. Syst.*, 2014, pp. 2672–2680.
- [27] W. Li, J. Chen, Z. Wang, Z. Shen, C. Ma, and X. Cui, "IFL-GAN: Improved federated learning generative adversarial network with maximum mean discrepancy model aggregation," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 12, pp. 10502–10515, Dec. 2023.
- [28] A. Wijesinghe, S. Zhang, and Z. Ding, "PFL-GAN: When client heterogeneity meets generative models in personalized federated learning," 2023, *arXiv:2308.12454*.
- [29] L. Yang et al., "Enhanced model poisoning attack and multi-strategy defense in federated learning," *IEEE Trans. Inf. Forensics Secur.*, vol. 20, pp. 3877–3892, 2025.

- [30] L. Melis, C. Song, E. De Cristofaro, and V. Shmatikov, "Exploiting unintended feature leakage in collaborative learning," in *Proc. IEEE Symp. Secur. Privacy*, 2019, pp. 691–706.
- [31] M. Nasr, R. Shokri, and A. Houmansadr, "Comprehensive privacy analysis of deep learning: Passive and active white-box inference attacks against centralized and federated learning," in *Proc. IEEE Symp. Secur. Privacy*, 2019, pp. 739–753.
- [32] J. Geiping, H. Bauermeister, H. Dröge, and M. Moeller, "Inverting gradients—How easy is it to break privacy in federated learning?," in *Proc. 34th Int. Conf. Neural Inf. Process. Syst.*, 2020, pp. 16937–16947.
- [33] C. Dwork, "Differential privacy," in *Proc. Int. Colloq. Automata, Languages, Program.*, 2006, pp. 1–12.
- [34] X. Huang, Y. Ding, Z. L. Jiang, S. Qi, X. Wang, and Q. Liao, "DP-FL: A novel differentially private federated learning framework for the unbalanced data," *World Wide Web*, vol. 23, pp. 2529–2545, 2020.
- [35] A. Salam, M. Abrar, F. Ullah, I. A. Khan, F. Amin, and G. S. Choi, "Efficient data collaboration using multi-party privacy preserving machine learning framework," *IEEE Access*, vol. 11, pp. 138151–138164, 2023.
- [36] K. Wei et al., "Federated learning with differential privacy: Algorithms and performance analysis," *IEEE Trans. Inf. Forensics Secur.*, vol. 15, pp. 3454–3469, 2020.
- [37] K. Wei et al., "Personalized federated learning with differential privacy and convergence guarantee," *IEEE Trans. Inf. Forensics Secur.*, vol. 18, pp. 4488–4503, 2023.
- [38] J. Du, J. Hu, Z. Wang, P. Sun, N. Z. Gong, and K. Ren, "SoK: Gradient leakage in federated learning," 2024, *arXiv:2404.05403*.
- [39] S. Zhao, Z. Liu, J. Lin, J.-Y. Zhu, and S. Han, "Differentiable augmentation for data-efficient GAN training," in *Proc. 34th Int. Conf. Neural Inf. Process. Syst.*, 2020, pp. 7559–7570.
- [40] Y. Qi, M. S. Hossain, J. Nie, and X. Li, "Privacy-preserving blockchain-based federated learning for traffic flow prediction," *Future Gener. Comput. Syst.*, vol. 117, pp. 328–337, 2021.
- [41] A. Radford, L. Metz, and S. Chintala, "Unsupervised representation learning with deep convolutional generative adversarial networks," in *Proc. Int. Conf. Learn. Representations*, San Juan, Puerto Rico, May 2016.
- [42] M. Arjovsky, S. Chintala, and L. Bottou, "Wasserstein generative adversarial networks," in *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 214–223.
- [43] X. Chen, Y. Duan, R. Houthoofd, J. Schulman, I. Sutskever, and P. Abbeel, "InfoGAN: Interpretable representation learning by information maximizing generative adversarial nets," in *Proc. Adv. Neural Inf. Process. Syst.*, 2016, pp. 2180–2188.
- [44] J. Wen, B.-Y. Chen, C.-D. Wang, and Z. Tian, "PRGAN: Personalized recommendation with conditional generative adversarial networks," in *Proc. IEEE Int. Conf. Data Mining*, 2021, pp. 729–738.
- [45] J. Hayes, L. Melis, G. Danezis, and E. De Cristofaro, "LOGAN: Membership inference attacks against generative models," in *Proc. Privacy Enhancing Technol.*, 2019, pp. 133–152.
- [46] C. Zheng, X. Jiang, Y. Zhang, X. Liu, B. Yuan, and Z. Li, "Self-normalizing generative adversarial network for super-resolution reconstruction of SAR images," in *Proc. IEEE Int. Geosci. Remote Sens. Symp.*, 2019, pp. 1911–1914.
- [47] C. Dwork et al., "The algorithmic foundations of differential privacy," *Foundations Trends Theor. Comput. Sci.*, vol. 9, pp. 211–407, 2014.
- [48] J. Wang, S. Guo, X. Xie, and H. Qi, "Protect privacy from gradient leakage attack in federated learning," in *Proc. IEEE INFOCOM IEEE Conf. Comput. Commun.*, 2022, pp. 580–589.
- [49] M. Tang and V. W. Wong, "Tackling system induced bias in federated learning: Stratification and convergence analysis," in *Proc. IEEE INFOCOM IEEE Conf. Comput. Commun.*, 2023, pp. 1–10.
- [50] Z. Li, J. Zhang, L. Liu, and J. Liu, "Auditing privacy defenses in federated learning via generative gradient leakage," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 10122–10132.
- [51] H. Chen and H. Vikalo, "Federated learning in non-IID settings aided by differentially private synthetic data," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 5027–5036.
- [52] J. Guo et al., "TFL-DT: A trust evaluation scheme for federated learning in digital twin for mobile networks," *IEEE J. Sel. Areas Commun.*, vol. 41, no. 11, pp. 3548–3560, Nov. 2023.
- [53] Y. Gu and Y. Bai, "LDIA: Label distribution inference attack against federated learning in edge computing," *J. Inf. Secur. Appl.*, vol. 74, 2023, Art. no. 103475.
- [54] A. Salem, A. Bhattacharya, M. Backes, M. Fritz, and Y. Zhang, "{Updates-Leak} : Data set inference and reconstruction attacks in online learning," in *Proc. 29th USENIX Secur. Symp.*, 2020, pp. 1291–1308.
- [55] R. Torkzadehmahani, P. Kairouz, and B. Paten, "DP-CGAN: Differentially private synthetic data and label generation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. Workshops*, 2019, pp. 98–104.
- [56] Y. Yu, W. Zhang, and Y. Deng, "Frechet inception distance (FID) for evaluating gans," China University of Mining Technology Beijing Graduate School, vol. 3, no. 11, 2021.
- [57] F. Zou, L. Shen, Z. Jie, W. Zhang, and W. Liu, "A sufficient condition for convergences of adam and RMSProp," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2019, pp. 11119–11127.
- [58] Z. Liu, J. Guo, W. Yang, J. Fan, K.-Y. Lam, and J. Zhao, "Privacy-preserving aggregation in federated learning: A survey," *IEEE Trans. Big Data*, vol. 9, no. 1, pp. 205–218, Feb. 2023, doi: [10.1109/TB-DATA.2022.3190835](https://doi.org/10.1109/TB-DATA.2022.3190835).
- [59] M. Abadi et al., "Deep learning with differential privacy," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2016, pp. 308–318.
- [60] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2324, Nov. 1998.
- [61] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-mnist: A novel image dataset for benchmarking machine learning algorithms," 2017, *arXiv:1708.07747*.
- [62] A. Krizhevsky et al., "Learning multiple layers of features from tiny images," Univ. Toronto, Techn. Rep., 2009.
- [63] Y. Netzer, T. Wang, A. Coates, A. Bissacco, B. Wu, and A. Y. Ng, "Reading digits in natural images with unsupervised feature learning," in *Proc. NIPS Workshop Deep Learn. Unsupervised Feature Learn.*, 2011, no. 5, pp. 7.
- [64] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proc. IEEE*, vol. 86, no. 11, pp. 2278–2323, Nov. 1998, doi: [10.1109/5.726791](https://doi.org/10.1109/5.726791).
- [65] C. Szegedy et al., "Going deeper with convolutions," in *Proc. IEEE Conf. Computer Vis. Pattern Recognit.*, 2015, pp. 1–9.
- [66] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2016, pp. 770–778.
- [67] H. Wang, M. Yurochkin, Y. Sun, D. Papailiopoulos, and Y. Khazaeni, "Federated learning with matched averaging," in *Proc. Int. Conf. Learn. Representations*, Apr. 2020.
- [68] S. P. Karimireddy, S. Kale, M. Mohri, S. Reddi, S. Stich, and A. T. Suresh, "SCAFFOLD: Stochastic controlled averaging for federated learning," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 5132–5143.
- [69] M. Asif et al., "Advanced zero-shot learning (AZSL) framework for secure model generalization in federated learning," *IEEE Access*, vol. 12, pp. 184393–184407, 2024.
- [70] Y. Cheng, L. Zhang, and A. Li, "GFL: Federated learning on non-IID data via privacy-preserving synthetic data," in *Proc. IEEE Int. Conf. Pervasive Comput. Commun.*, 2023, pp. 61–70.
- [71] W. Hao et al., "Towards fair federated learning with zero-shot data augmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 3305–3314.
- [72] P. J. Maliakel, S. Ilager, and I. Brandic, "FLIGAN: Enhancing federated learning with incomplete data using GAN," in *Proc. 7th Int. Workshop Edge Syst., Analytics Netw.*, 2024, pp. 1–6.
- [73] T. Li, S. Hu, A. Beirami, and V. Smith, "Ditto: Fair and robust federated learning through personalization," in *Proc. Int. Conf. Mach. Learn.*, 2021, pp. 6357–6368.
- [74] T. Yoon, S. Shin, S. J. Hwang, and E. Yang, "Fedmix: Approximation of mixup under mean augmented federated learning," in *Proc. Int. Conf. Learn. Representations*, May 2021.
- [75] J. Zhang, L. Zhao, K. Yu, G. Min, A. Y. Al-Dubai, and A. Y. Zomaya, "A novel federated learning scheme for generative adversarial networks," *IEEE Trans. Mobile Comput.*, vol. 23, no. 5, pp. 3633–3649, May 2024.
- [76] P. Mohassel and Y. Zhang, "SecureML: A system for scalable privacy-preserving machine learning," in *Proc. IEEE Symp. Secur. Privacy*, 2017, pp. 19–38.
- [77] K. Bonawitz et al., "Practical secure aggregation for privacy-preserving machine learning," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2017, pp. 1175–1191.
- [78] C. Zhang, S. Li, J. Xia, W. Wang, F. Yan, and Y. Liu, "{BatchCrypt}: Efficient homomorphic encryption for {Cross-Silo} federated learning," in *Proc. 2020 USENIX Annu. Techn. Conf.*, 2020, pp. 493–506.
- [79] Y. Aono et al., "Privacy-preserving deep learning via additively homomorphic encryption," *IEEE Trans. Inf. Forensics Secur.*, vol. 13, no. 5, pp. 1333–1345, May 2018.

- [80] R. W. Anwar, M. Abrar, A. Salam, and F. Ullah, "Federated learning with LSTM for intrusion detection in iot-based wireless sensor networks: A multi-dataset analysis," *PeerJ Comput. Sci.*, vol. 11, 2025.



Hao Bai received the BE degree in information security from Wuhan University, Wuhan, China, in 2024, where he is currently working toward the Ph.D. degree with the School of Cyber Science and Engineering. His research interests include distributed machine learning.



Haowei Li is currently working toward the BE degree in information security from Wuhan University, Wuhan, China. His research interests include federated learning.



Zhongmou Liu is currently working toward the BE degree in information security from Wuhan University, Wuhan, China. His research interests include federated learning.



Kun He (Member, IEEE) received the PhD degree in computer science with the computer school, Wuhan University, Wuhan, China, where he is currently an associate professor. He has authored or coauthored more than 20 research papers in many international journals and conferences, such as *IEEE Transactions on Computers*, *IEEE Transactions on Dependable and Secure Computing*, *IEEE Transactions on Mobile Computing*, *IEEE Transactions on Parallel and Distributed Systems*, *USENIX Security*, *CCS*, and *INFOCOM*. His research interests include cryptography,

network security, mobile computing, and cloud computing.



Xuanang Yang received the BE degree in software engineering from Wuhan University, Wuhan, China, in 2019, and the PhD degree with the School of Cyber Science and Engineering, in 2025. His research interests include privacy-preserving machine learning and distributed machine learning.



Yuqing Li (Member, IEEE) received the PhD degree in electronic engineering from Shanghai Jiao Tong University, Shanghai, China, in 2019. She is currently an associate professor with the School of Cyber Science and Engineering, Wuhan University, Wuhan, China. From 2019 to 2020, she was a postdoctoral fellow with the Hong Kong University of Science and Technology, Hong Kong. From 2020 to 2022, she was a researcher with Huawei Hong Kong Research Center, Hong Kong. Her research interests include data privacy and security, distributed machine

learning, and edge computing.



Ruiying Du received the BS, MS, PhD degrees in computer science in 1987, 1994, and 2008, respectively, from Wuhan University, Wuhan, China. She is a professor with the School of Cyber Science and Engineering, Wuhan University. She has authored or coauthored more than 80 research papers in many international journals and conferences, such as *IEEE Transactions on Parallel and Distributed System*, *International Journal of Parallel and Distributed System*, *INFOCOM*, *SECON*, *TrustCom*, *NSS*. Her research interests include network security, wireless

network, cloud computing and mobile computing.



Jing Chen (Senior Member, IEEE) received the PhD degree in computer science from the Huazhong University of Science and Technology, Wuhan, China. He was a full professor with Wuhan University from 2015. He has authored or coauthored more than 100 research papers in many international journals and conferences, such as *IEEE Transactions on Dependable and Secure Computing*, *IEEE Transactions on Information Forensics and Security*, *IEEE Transactions on Mobile Computing*, *IEEE Transactions on Computers*, *IEEE Transactions on Parallel and*

Distributed Systems, *USENIX Security*, *CCS*, and *INFOCOM*. His research interests in computer science are in the areas of network security, cloud security. He is also a reviewer for many journals and conferences, such as *IEEE Transactions on Information Forensics*, *IEEE Transactions on Computers*, *IEEE/ACM Transactions on Networking*.